

École Militaire Polytechnique

Département d'Informatique

Module : Sécurité Réseau

Rapport de TP

Linux Firewall Exploration Lab

Nom	Bakouche
Prénom	Mahmoud
Groupe	RSD
Année universitaire	2025/2026

13 avril 2026

Table des matières

1	Introduction et environnement	2
2	Task 1 : Utilisation de iptables	2
2.1	Objectif	2
2.2	Règles appliquées	2
2.3	Résultats	2
3	Task 2 : Pare-feu avec LKM et Netfilter	4
3.1	Objectif	4
3.2	Principe	4
3.3	Règles implémentées	4
3.4	Résultats	5
4	Task 3 : Contourner le filtrage sortant (Egress Filtering)	9
4.1	Objectif	9
4.2	Task 3.a — Telnet via tunnel SSH local	9
4.3	Task 3.b — Navigation web via proxy SOCKS dynamique	9
4.4	Résultats	10
5	Task 4 : Contourner le filtrage entrant (Ingress Filtering)	16
5.1	Objectif	16
5.2	Principe	16
5.3	Résultats	16
6	Conclusion	17

1. Introduction et environnement

Ce TP explore le filtrage de paquets sous Linux via `iptables` et un module noyau Netfilter (LKM), puis montre comment des tunnels SSH permettent de contourner ces filtrages.

Machines utilisées :

- Machine A : 192.168.56.10 — machine filtrée
- Machine B : 192.168.56.11 — machine externe / relais SSH
- Machine C : 192.168.56.12 — serveur Telnet
- Site bloqué : 103.224.182.245

2. Task 1 : Utilisation de iptables

2.1. Objectif

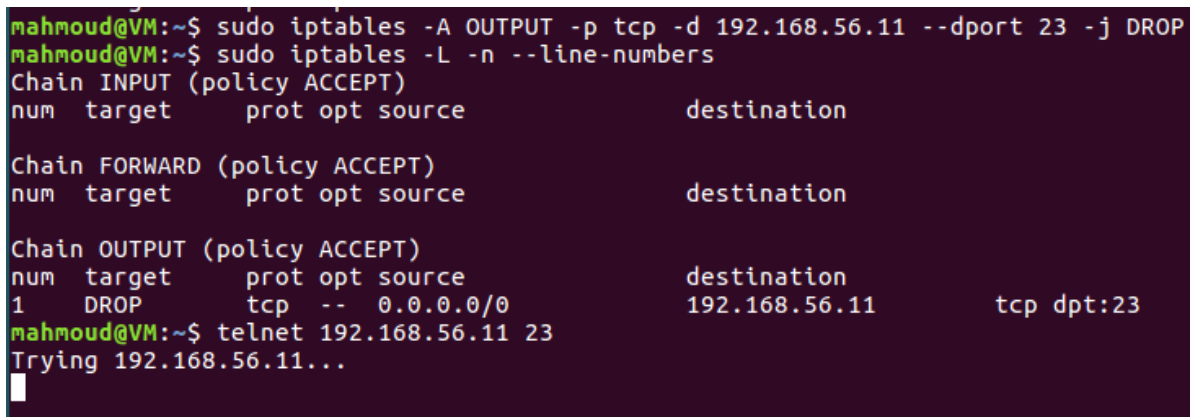
Configurer `iptables` sur Machine A pour : bloquer le Telnet entre A et B, et bloquer l'accès HTTP/HTTPS vers un site externe.

2.2. Règles appliquées

```
sudo iptables -A OUTPUT -p tcp -d 192.168.56.11 --dport 23 -j DROP
sudo iptables -A INPUT -p tcp -s 192.168.56.11 --dport 23 -j DROP
sudo iptables -A OUTPUT -p tcp -d 103.224.182.245 --dport 80 -j DROP
sudo iptables -A OUTPUT -p tcp -d 103.224.182.245 --dport 443 -j DROP
```

La chaîne `OUTPUT` filtre le trafic sortant de A, la chaîne `INPUT` filtre le trafic entrant. L'option `-j DROP` supprime silencieusement le paquet.

2.3. Résultats



```
mahmoud@VM:~$ sudo iptables -A OUTPUT -p tcp -d 192.168.56.11 --dport 23 -j DROP
mahmoud@VM:~$ sudo iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination

Chain FORWARD (policy ACCEPT)
num target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
num target      prot opt source                destination
1 DROP         tcp  --  0.0.0.0/0             192.168.56.11         tcp dpt:23
mahmoud@VM:~$ telnet 192.168.56.11 23
Trying 192.168.56.11...
```

FIGURE 1 – Ajout de la règle bloquant le Telnet sortant (A → B) et affichage des règles `iptables`.

```

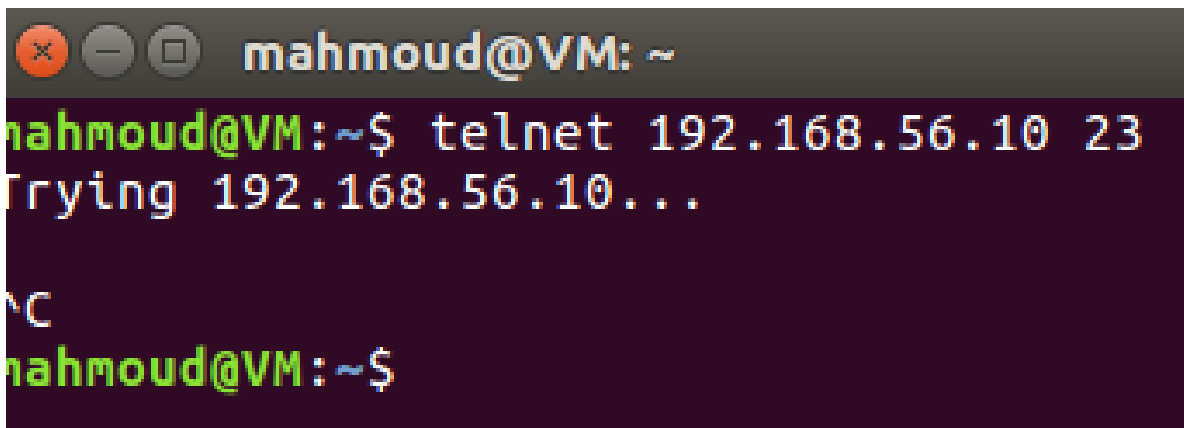
mahmoud@VM:~$ sudo iptables -A INPUT -p tcp -s 192.168.56.11 --dport 23 -j DROP
mahmoud@VM:~$ sudo iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num  target      prot opt source                destination            tcp dpt:23
1    DROP        tcp  --  192.168.56.11          0.0.0.0/0              tcp dpt:23

Chain FORWARD (policy ACCEPT)
num  target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
num  target      prot opt source                destination            tcp dpt:23
1    DROP        tcp  --  0.0.0.0/0              192.168.56.11          tcp dpt:23
mahmoud@VM:~$

```

FIGURE 2 – Échec de la connexion Telnet de A vers B après application du filtrage.



```

mahmoud@VM:~$ telnet 192.168.56.10 23
Trying 192.168.56.10...
^C
mahmoud@VM:~$

```

FIGURE 3 – Règle bloquant le Telnet entrant (B → A) et affichage de la chaîne INPUT.

```

mahmoud@VM:~$ curl http://exemple.com
<html>
<head>
<title>exemple.com</title>
<script type="text/javascript" src="/js/fingerprint/iife.min.js"></script>
<script type="text/javascript">
var redirect_link = 'http://exemple.com/?tr_uuid=20260413-1042-171e-abbd-3b0e3f1
b768c&';
let redirected = false;

function redirect(suffix) {
    if (redirected) return;
    redirected = true;
    window.location.replace(redirect_link + suffix);
}

```

FIGURE 4 – Accès au site externe avant blocage (test de référence).

```

mahmoud@VM:~$ sudo iptables -A OUTPUT -p tcp -d 103.224.182.245 --dport 443 -j DROP
mahmoud@VM:~$ sudo iptables -A OUTPUT -p tcp -d 103.224.182.245 --dport 80 -j DROP
mahmoud@VM:~$ sudo iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num  target      prot opt source                destination
1    DROP        tcp  --  192.168.56.11          0.0.0.0/0            tcp dpt:23

Chain FORWARD (policy ACCEPT)
num  target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
num  target      prot opt source                destination
1    DROP        tcp  --  0.0.0.0/0              192.168.56.11        tcp dpt:23
2    DROP        tcp  --  0.0.0.0/0              103.224.182.245      tcp dpt:443
3    DROP        tcp  --  0.0.0.0/0              103.224.182.245      tcp dpt:80
mahmoud@VM:~$ curl http://exemple.com

```

FIGURE 5 – Blocage HTTP/HTTPS vers le site externe — l'accès échoue après ajout des règles.

3. Task 2 : Pare-feu avec LKM et Netfilter

3.1. Objectif

Implémenter un module noyau chargeable (LKM) qui utilise les hooks Netfilter pour filtrer les paquets directement dans le noyau, sans modifier le noyau lui-même.

3.2. Principe

Netfilter expose des points d'accroche (*hooks*) dans le chemin de traitement des paquets. Notre module s'y connecte via deux structures `nf_hook_ops` :

- `NF_INET_LOCAL_OUT` — paquets sortants
- `NF_INET_LOCAL_IN` — paquets entrants

Chaque hook inspecte les en-têtes IP, TCP ou ICMP et retourne `NF_DROP` ou `NF_ACCEPT`.

3.3. Règles implémentées

1. Bloquer Telnet A → B (port 23 sortant)
2. Bloquer Telnet B → A (port 23 entrant)
3. Bloquer HTTP A → site bloqué (port 80 sortant)
4. Bloquer HTTPS A → site bloqué (port 443 sortant)
5. Bloquer ping ICMP B → A (entrant)

3.4. Résultats

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/init.h>
#include <linux/netfilter.h>
#include <linux/netfilter_ipv4.h>
#include <linux/ip.h>
#include <linux/tcp.h>
#include <linux/icmp.h>
#include <linux/in.h>
#include <linux/inet.h>

MODULE_LICENSE("GPL");

#define IP_A      in_aton("192.168.56.10")
#define IP_B      in_aton("192.168.56.11")
#define IP_BLOCKED in_aton("103.224.182.245")

static struct nf_hook_ops hook_in;
static struct nf_hook_ops hook_out;
```

FIGURE 6 – Déclarations des adresses IP, includes et structures Netfilter du module.

```
/* Paquets entrants */
static unsigned int fw_in(void *priv,
                           struct sk_buff *skb,
                           const struct nf_hook_state *state)
{
    struct iphdr *ip_header;
    struct tcphdr *tcp_header;
    struct icmp_hdr *icmp_header;

    if (!skb)
        return NF_ACCEPT;

    ip_header = ip_hdr(skb);
    if (!ip_header)
        return NF_ACCEPT;

    /* Bloquer B -> A en telnet */
    if (ip_header->protocol == IPPROTO_TCP) {
        tcp_header = tcp_hdr(skb);

        if (ip_header->saddr == IP_B &&
            ip_header->daddr == IP_A &&
            ntohs(tcp_header->dest) == 23) {
            printk(KERN_INFO "[FW] DROP B -> A telnet\n");
            return NF_DROP;
        }
    }

    /* Bloquer ping de B vers A */
    if (ip_header->protocol == IPPROTO_ICMP) {
        icmp_header = icmp_hdr(skb);

        if (ip_header->saddr == IP_B &&
            ip_header->daddr == IP_A &&
            icmp_header->type == ICMP_ECHO) {
            printk(KERN_INFO "[FW] DROP B -> A ping\n");
            return NF_DROP;
        }
    }
}
```

FIGURE 7 – Fonction `fw_in()` — filtrage des paquets entrants (Telnet et ICMP).

```

/* Paquets sortants */
static unsigned int fw_out(void *priv,
                           struct sk_buff *skb,
                           const struct nf_hook_state *state)
{
    struct iphdr *ip_header;
    struct tcphdr *tcp_header;

    if (!skb)
        return NF_ACCEPT;

    ip_header = ip_hdr(skb);
    if (!ip_header)
        return NF_ACCEPT;

    if (ip_header->protocol == IPPROTO_TCP) {
        tcp_header = tcp_hdr(skb);

        /* Bloquer A -> B en telnet */
        if (ip_header->saddr == IP_A &&
            ip_header->daddr == IP_B &&
            ntohs(tcp_header->dest) == 23) {
            printk(KERN_INFO "[FW] DROP A -> B telnet\n");
            return NF_DROP;
        }

        /* Bloquer A -> site web en HTTP */
        if (ip_header->daddr == IP_BLOCKED &&
            ntohs(tcp_header->dest) == 80) {
            printk(KERN_INFO "[FW] DROP A -> blocked web HTTP\n");
            return NF_DROP;
        }

        /* Bloquer A -> site web en HTTPS */
        if (ip_header->daddr == IP_BLOCKED &&
            ntohs(tcp_header->dest) == 443) {
            printk(KERN_INFO "[FW] DROP A -> blocked web HTTPS\n");
            return NF_DROP;
        }
    }
}

```

FIGURE 8 – Fonction fw_out() — filtrage des paquets sortants (Telnet, HTTP, HTTPS).

```
static int __init firewall_init(void)
{
    printk(KERN_INFO "[FW] Firewall module loaded\n");

    hook_in.hook = fw_in;
    hook_in.hooknum = NF_INET_LOCAL_IN;
    hook_in.pf = PF_INET;
    hook_in.priority = NF_IP_PRI_FIRST;
    nf_register_net_hook(&init_net, &hook_in);

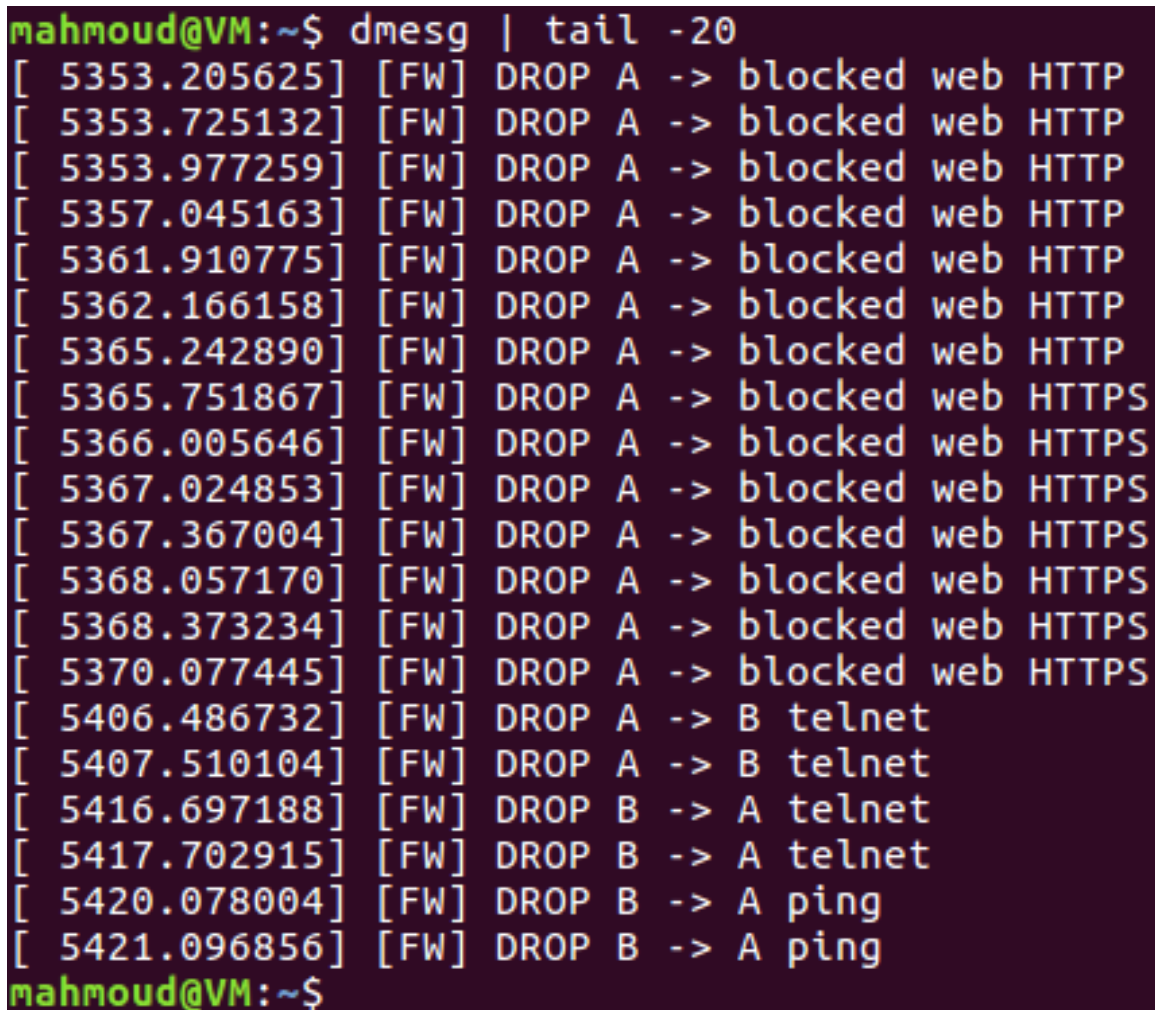
    hook_out.hook = fw_out;
    hook_out.hooknum = NF_INET_LOCAL_OUT;
    hook_out.pf = PF_INET;
    hook_out.priority = NF_IP_PRI_FIRST;
    nf_register_net_hook(&init_net, &hook_out);

    return 0;
}

static void __exit firewall_exit(void)
{
    nf_unregister_net_hook(&init_net, &hook_in);
    nf_unregister_net_hook(&init_net, &hook_out);
    printk(KERN_INFO "[FW] Firewall module removed\n");
}

module_init(firewall_init);
module_exit(firewall_exit);
```

FIGURE 9 – Initialisation et libération du module avec enregistrement des hooks.

A terminal window with a dark background and light-colored text. The prompt is 'mahmoud@VM:~\$'. The command 'dmesg | tail -20' has been executed. The output shows 20 lines of kernel messages. The first 13 lines are about blocked web traffic (HTTP and HTTPS) with timestamps ranging from 5353.205625 to 5370.077445. The next 4 lines are about blocked telnet traffic with timestamps 5406.486732 to 5417.702915. The last 3 lines are about blocked ping traffic with timestamps 5420.078004 and 5421.096856. The prompt 'mahmoud@VM:~\$' is visible at the bottom.

```
mahmoud@VM:~$ dmesg | tail -20
[ 5353.205625] [FW] DROP A -> blocked web HTTP
[ 5353.725132] [FW] DROP A -> blocked web HTTP
[ 5353.977259] [FW] DROP A -> blocked web HTTP
[ 5357.045163] [FW] DROP A -> blocked web HTTP
[ 5361.910775] [FW] DROP A -> blocked web HTTP
[ 5362.166158] [FW] DROP A -> blocked web HTTP
[ 5365.242890] [FW] DROP A -> blocked web HTTP
[ 5365.751867] [FW] DROP A -> blocked web HTTPS
[ 5366.005646] [FW] DROP A -> blocked web HTTPS
[ 5367.024853] [FW] DROP A -> blocked web HTTPS
[ 5367.367004] [FW] DROP A -> blocked web HTTPS
[ 5368.057170] [FW] DROP A -> blocked web HTTPS
[ 5368.373234] [FW] DROP A -> blocked web HTTPS
[ 5370.077445] [FW] DROP A -> blocked web HTTPS
[ 5406.486732] [FW] DROP A -> B telnet
[ 5407.510104] [FW] DROP A -> B telnet
[ 5416.697188] [FW] DROP B -> A telnet
[ 5417.702915] [FW] DROP B -> A telnet
[ 5420.078004] [FW] DROP B -> A ping
[ 5421.096856] [FW] DROP B -> A ping
mahmoud@VM:~$
```

FIGURE 10 – dmesg confirmant le chargement du module et les paquets rejetés.

4. Task 3 : Contourner le filtrage sortant (Egress Filtering)

4.1. Objectif

Montrer qu'un filtrage sortant basé sur l'IP/port peut être contourné par encapsulation du trafic dans un tunnel SSH.

4.2. Task 3.a — Telnet via tunnel SSH local

Le pare-feu bloque tout Telnet sortant. On crée un tunnel SSH avec redirection de port :

```
ssh -L 8000:192.168.56.12:23 mahmoud@192.168.56.11
telnet localhost 8000
```

Le trafic Telnet est encapsulé dans SSH (port 22) vers B, qui le redirige vers C :23. Le pare-feu ne voit qu'une connexion SSH autorisée.

4.3. Task 3.b — Navigation web via proxy SOCKS dynamique

Le trafic HTTP/HTTPS est bloqué. On établit un tunnel dynamique et configure Firefox :

```
ssh -D 9000 -C mahmoud@192.168.56.11
```

Firefox utilise localhost:9000 comme proxy SOCKS v5. Toutes les requêtes transitent par le tunnel SSH chiffré vers B, qui contacte le site final.

4.4. Résultats

```
mahmoud@VM:~$ telnet 192.168.56.12 23
Trying 192.168.56.12...
Connected to 192.168.56.12.
Escape character is '^]'.
Ubuntu 16.04.2 LTS
VM login: mahmoud
Password:
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

mahmoud@VM:~$ exit
```

FIGURE 11 – Connexion Telnet directe vers C avant blocage (référence).

```
mahmoud@VM:~$ sudo iptables -A OUTPUT -p tcp --dport 23 -j DROP
mahmoud@VM:~$ sudo iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num target prot opt source destination

Chain FORWARD (policy ACCEPT)
num target prot opt source destination

Chain OUTPUT (policy ACCEPT)
num target prot opt source destination
1 DROP tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:23
mahmoud@VM:~$ telnet 192.168.56.12 23
Trying 192.168.56.12...
^C
mahmoud@VM:~$
```

FIGURE 12 – Règle iptables bloquant tout Telnet sortant depuis A.

```
mahmoud@VM:~$ ssh -L 8000:192.168.56.12:23 mahmoud@192.168.56.11
mahmoud@192.168.56.11's password:
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

Last login: Sun Apr 12 20:23:59 2026 from 192.168.56.10
mahmoud@VM:~$
```

FIGURE 13 – Échec du Telnet direct après activation du blocage.

```

mahmoud@VM:~$ telnet 192.168.56.12 23
Trying 192.168.56.12...
Connected to 192.168.56.12.
Escape character is '^]'.
Ubuntu 16.04.2 LTS
VM login: mahmoud*
Password:

Login incorrect
VM login: mahmoud
Password:

Last login: Mon Apr 13 02:38:21 EDT 2026 from 192.168.56.11 on pts/4
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

mahmoud@VM:~$ mahmoud

```

FIGURE 14 – Établissement du tunnel SSH local -L 8000:192.168.56.12:23.

No.	Time	Source	Destination	Protocol	Length	Info
255	353.439775084	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
256	353.442278642	192.168.56.11	192.168.56.12	TCP	66	59296 → 23 [ACK] Seq=3482954651 Ack=2107155293 Win=30336 Len=0 TSval=2167452 TSecr=26...
257	353.612374090	192.168.56.11	192.168.56.12	TELNET	67	Telnet Data ...
258	353.613527644	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
259	353.619047053	192.168.56.11	192.168.56.12	TCP	66	59296 → 23 [ACK] Seq=3482954652 Ack=2107155294 Win=30336 Len=0 TSval=2167495 TSecr=26...
260	353.863703454	192.168.56.11	192.168.56.12	TELNET	67	Telnet Data ...
261	353.864581109	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
262	353.867843456	192.168.56.11	192.168.56.12	TCP	66	59296 → 23 [ACK] Seq=3482954653 Ack=2107155295 Win=30336 Len=0 TSval=2167558 TSecr=26...
263	354.049697909	192.168.56.11	192.168.56.12	TELNET	67	Telnet Data ...
264	354.051738519	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
265	354.053654104	192.168.56.11	192.168.56.12	TCP	66	59296 → 23 [ACK] Seq=3482954654 Ack=2107155296 Win=30336 Len=0 TSval=2167605 TSecr=26...
266	354.222175690	192.168.56.11	192.168.56.12	TELNET	67	Telnet Data ...
267	354.222879463	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
268	354.224608527	192.168.56.11	192.168.56.12	TCP	66	59296 → 23 [ACK] Seq=3482954655 Ack=2107155297 Win=30336 Len=0 TSval=2167648 TSecr=26...
269	354.386902133	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
270	354.388006279	192.168.56.12	192.168.56.11	TELNET	67	Telnet Data ...
271	354.392304943	192.168.56.11	192.168.56.12	TCP	66	59296 → 23 [ACK] Seq=3482954656 Ack=2107155298 Win=30336 Len=0 TSval=2167689 TSecr=26...

▶	Frame 270: 67 bytes on wire (536 bits), 67 bytes captured (536 bits) on interface 0
▶	Ethernet II, Src: PcsCompu_c1:ae:23 (08:00:27:c1:ae:23), Dst: PcsCompu_83:4f:4e (08:00:27:83:4f:4e)
▶	Internet Protocol Version 4, Src: 192.168.56.12, Dst: 192.168.56.11
▶	Transmission Control Protocol, Src Port: 23, Dst Port: 59296, Seq: 2107155297, Ack: 3482954656, Len: 1
▶	Telnet

0000	08 00 27 83 4f 4e 08 00	27 c1 ae 23 08 00 45 10	..'.ON..'.#...E.
0010	00 35 44 13 40 00 40 06	05 38 c0 a8 38 0c c0 a8	.5D.@.0. .8...8...
0020	38 0b 00 17 e7 a0 7d 98	a3 61 cf 99 ab a0 80 18	8.....}.a.....
0030	00 e3 f1 8f 00 00 01 01	08 0a 00 04 1e 6a 00 21	j..!
0040	13 88 64		..d

FIGURE 15 – Connexion Telnet réussie via localhost:8000 à travers le tunnel.

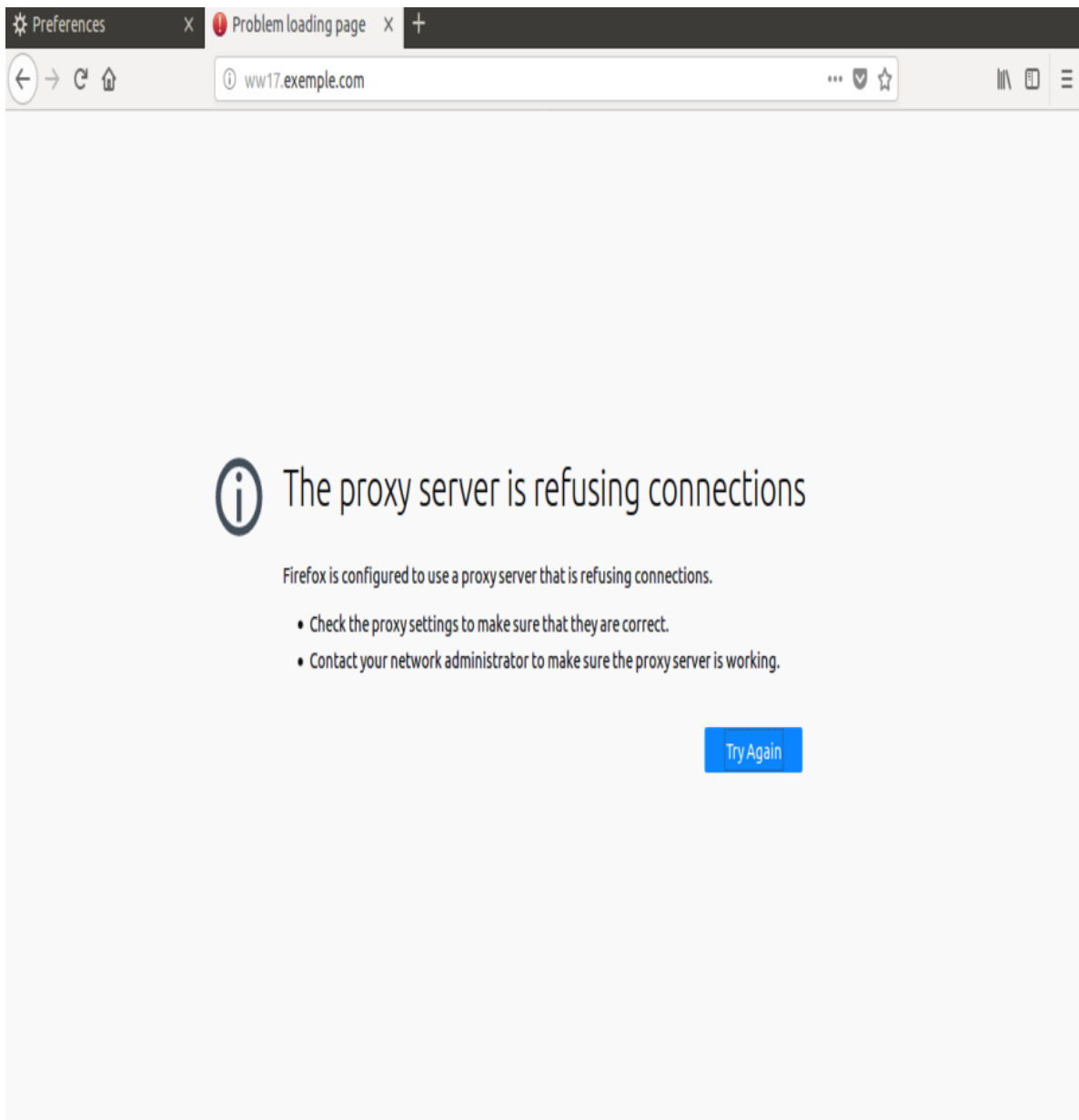


FIGURE 16 – Wireshark : trafic Telnet observé entre B et C via le tunnel.

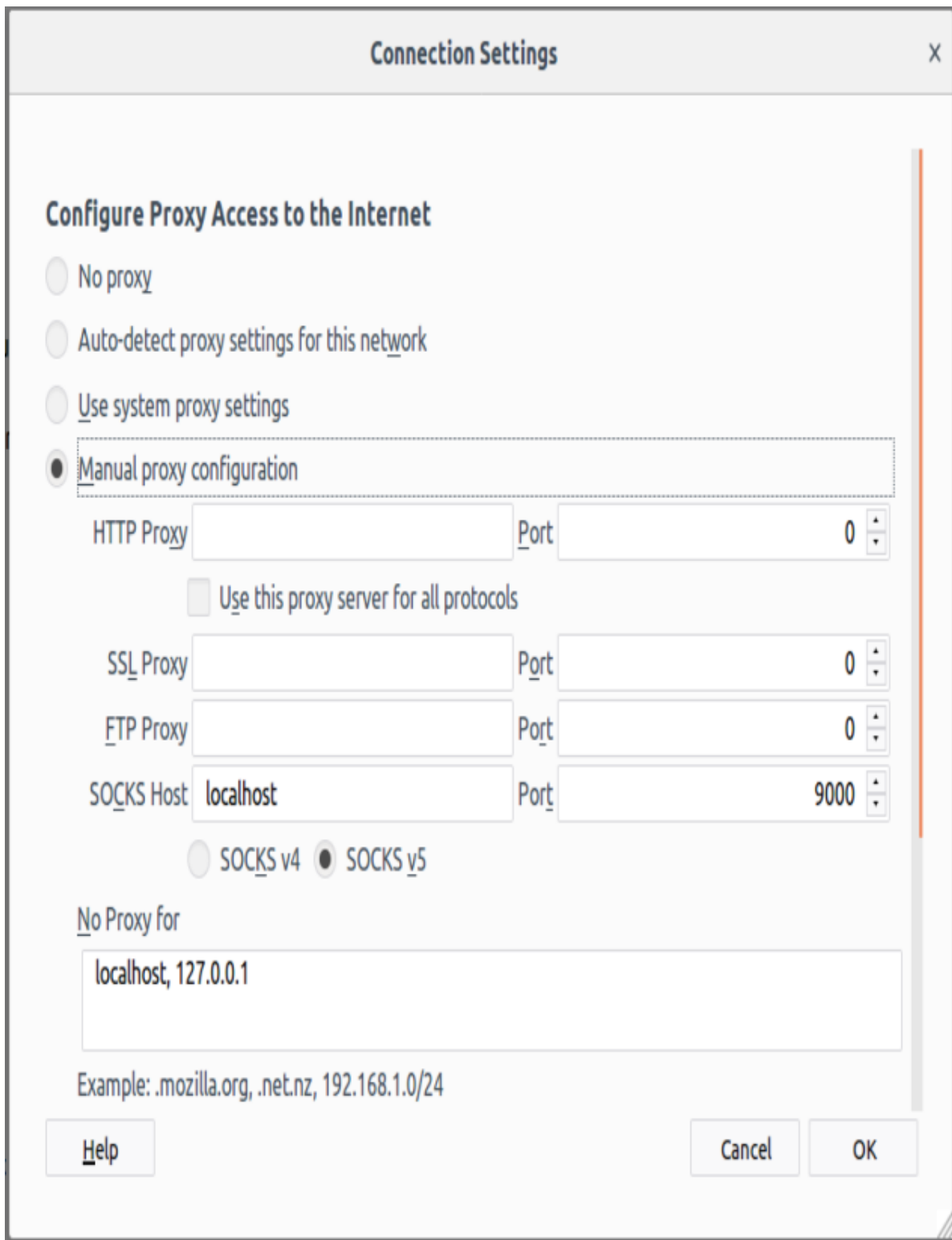
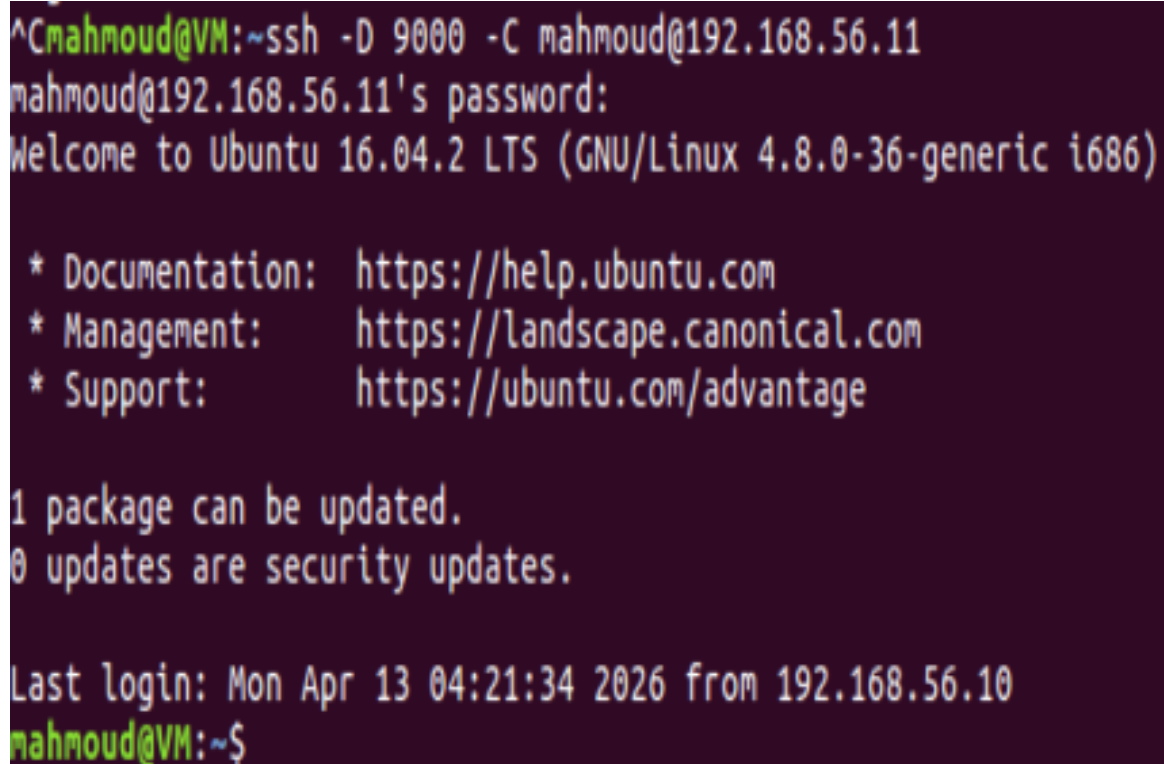


FIGURE 17 — Firefox sans proxy — message d'erreur d'accès au site bloqué.

A terminal window showing an SSH session. The user 'mahmoud' is logged into a VM. The prompt is 'mahmoud@VM:~\$'. The terminal output shows the SSH command, the password prompt, the Ubuntu 16.04.2 LTS welcome message, system information, update notifications, and the last login time.

```
^Cmahmoud@VM:~ssh -D 9000 -C mahmoud@192.168.56.11
mahmoud@192.168.56.11's password:
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

Last login: Mon Apr 13 04:21:34 2026 from 192.168.56.10
mahmoud@VM:~$
```

FIGURE 18 – Configuration du proxy SOCKS v5 dans Firefox (localhost:9000).

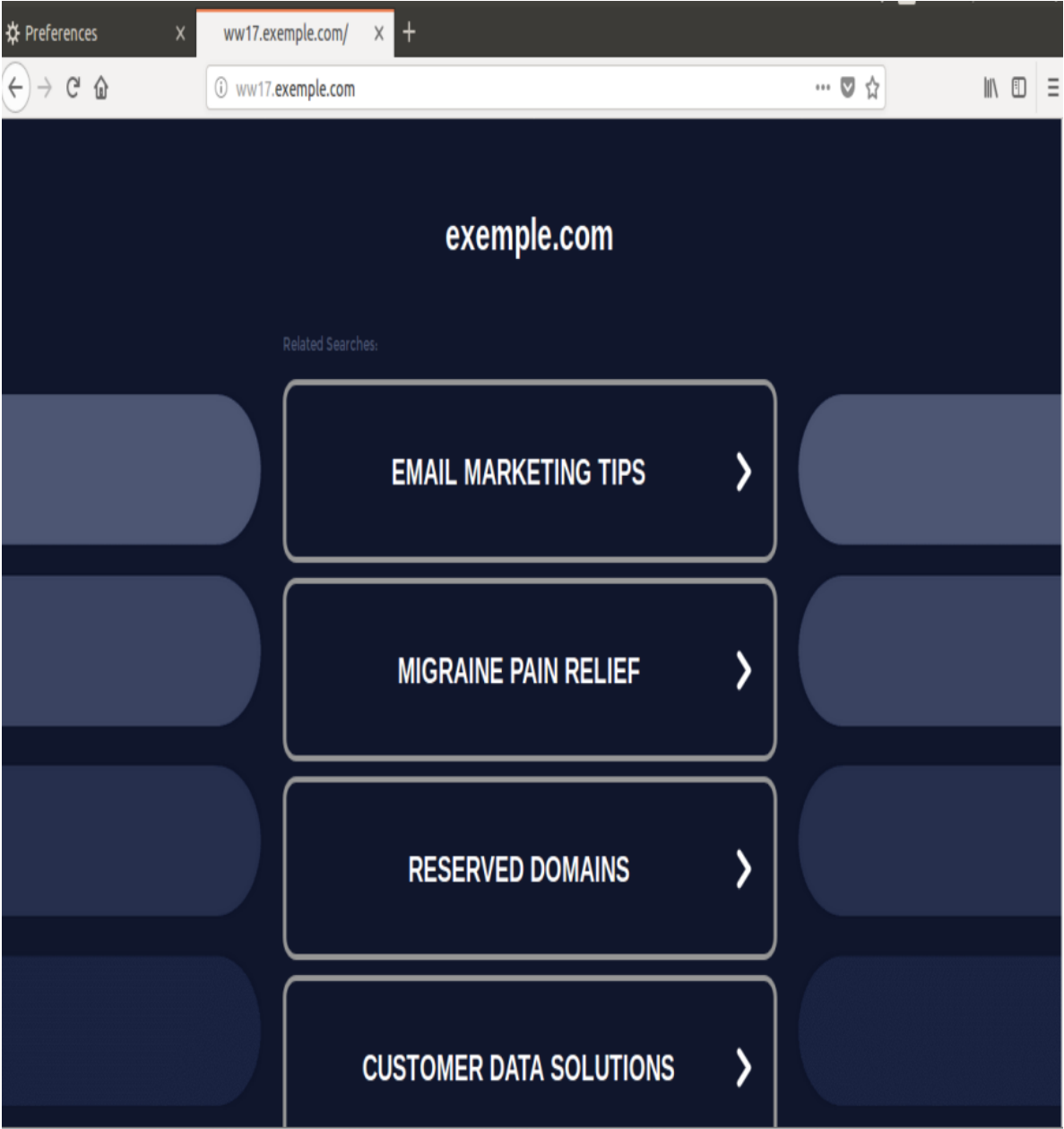


FIGURE 19 – Établissement du tunnel SSH dynamique -D 9000 -C.

512	66.465896697	192.168.56.11	192.168.56.10	SSH	142 Server: Encrypted packet (len=76)
513	66.465913654	192.168.56.10	192.168.56.11	TCP	66 38144 → 22 [ACK] Seq=2721262559 Ack=2129106832 Win=1324 Len=0 TSval=2993765 TSecr=280...
514	84.176181526	192.168.56.10	192.168.56.11	SSH	118 Client: Encrypted packet (len=52)
515	84.177634408	192.168.56.11	192.168.56.10	TCP	66 22 → 38144 [ACK] Seq=2129106832 Ack=2721262611 Win=940 Len=0 TSval=2808772 TSecr=2998...
516	84.182065830	192.168.56.11	192.168.56.10	SSH	118 Server: Encrypted packet (len=52)
517	84.182082914	192.168.56.10	192.168.56.11	TCP	66 38144 → 22 [ACK] Seq=2721262611 Ack=2129106884 Win=1324 Len=0 TSval=2998194 TSecr=280...
518	84.909078929	192.168.56.10	192.168.56.11	SSH	102 Client: Encrypted packet (len=36)
519	84.909702386	192.168.56.11	192.168.56.10	SSH	102 Server: Encrypted packet (len=36)
520	84.909734702	192.168.56.10	192.168.56.11	TCP	66 38144 → 22 [ACK] Seq=2721262647 Ack=2129106920 Win=1324 Len=0 TSval=2998376 TSecr=280...
521	85.352467001	192.168.56.11	192.168.56.10	SSH	134 Server: Encrypted packet (len=68)
522	85.352498892	192.168.56.10	192.168.56.11	TCP	66 38144 → 22 [ACK] Seq=2721262647 Ack=2129106988 Win=1324 Len=0 TSval=2998486 TSecr=280...
523	85.354116938	192.168.56.11	192.168.56.10	SSH	142 Server: Encrypted packet (len=76)
524	85.354128844	192.168.56.10	192.168.56.11	TCP	66 38144 → 22 [ACK] Seq=2721262647 Ack=2129107064 Win=1324 Len=0 TSval=2998487 TSecr=280...
525	85.355736700	192.168.56.11	192.168.56.10	SSH	102 Server: Encrypted packet (len=36)
526	85.355746894	192.168.56.10	192.168.56.11	TCP	66 38144 → 22 [ACK] Seq=2721262647 Ack=2129107100 Win=1324 Len=0 TSval=2998487 TSecr=280...

FIGURE 20 – Accès réussi au site web grâce au proxy SOCKS et au tunnel SSH.

No.	Time	Source	Destination	Protocol	Length	Info
69	45.226656747	192.168.56.11	192.168.56.12	TCP	68	59442 → 23 [ACK] Seq=2805250579 Ack=259136436
70	45.229731981	10.0.2.15	103.224.182.245	TCP	76	41572 → 80 [SYN] Seq=2019412862 Win=29200 Len=0
71	45.435732111	103.224.182.245	10.0.2.15	TCP	62	80 → 41572 [SYN, ACK] Seq=238272001 Ack=2019412862
72	45.435768041	10.0.2.15	103.224.182.245	TCP	56	41572 → 80 [ACK] Seq=2019412863 Ack=238272002
73	45.436693148	10.0.2.15	103.224.182.245	HTTP	135	GET / HTTP/1.1
74	45.437576962	103.224.182.245	10.0.2.15	TCP	62	80 → 41572 [ACK] Seq=238272002 Ack=2019412942
75	45.684798771	103.224.182.245	10.0.2.15	TCP	226	[TCP segment of a reassembled PDU]
76	45.684831871	10.0.2.15	103.224.182.245	TCP	56	41572 → 80 [ACK] Seq=2019412942 Ack=238272172
77	45.685052646	103.224.182.245	10.0.2.15	HTTP	62	HTTP/1.0 403 Forbidden (text/html)
78	45.685970132	192.168.56.12	192.168.56.11	TELNET	102	Telnet Data ...
79	45.687495922	10.0.2.15	103.224.182.245	TCP	56	41572 → 80 [FIN, ACK] Seq=2019412942 Ack=238272172
80	45.688846392	103.224.182.245	10.0.2.15	TCP	62	80 → 41572 [ACK] Seq=238272173 Ack=2019412943
81	45.689408012	192.168.56.11	192.168.56.12	TCP	68	59442 → 23 [ACK] Seq=2805250579 Ack=259136436
82	45.689424436	192.168.56.12	192.168.56.11	TELNET	132	Telnet Data ...
83	45.690214616	192.168.56.11	192.168.56.12	TCP	68	59442 → 23 [ACK] Seq=2805250579 Ack=259136446
84	45.692262591	192.168.56.12	192.168.56.11	TELNET	126	Telnet Data ...
85	45.693596526	192.168.56.11	192.168.56.12	TCP	68	59442 → 23 [ACK] Seq=2805250579 Ack=259136446

FIGURE 21 – Wireshark : seul du trafic SSH chiffré est visible entre A et B.

5. Task 4 : Contourner le filtrage entrant (Ingress Filtering)

5.1. Objectif

Machine A héberge un serveur web. B ne peut pas accéder aux ports 22 et 80 de A (bloqués). Rendre le serveur accessible depuis B en utilisant un tunnel SSH inversé.

5.2. Principe

A initie une connexion SSH *sortante* vers B (autorisée par le pare-feu) et demande à B d'ouvrir le port 8080 redirigé vers `localhost:80` sur A :

```
# Sur Machine A :
ssh -R 8080:localhost:80 mahmoud@192.168.56.11

# Sur Machine B, pour accéder au serveur web de A :
curl http://localhost:8080
```

Le pare-feu bloque les connexions entrantes, mais laisse passer les connexions sortantes de A. Le tunnel exploite cette connexion autorisée pour faire passer le trafic HTTP dans le sens inverse.

5.3. Résultats

```
^Cmahmoud@VM:~$ sudo iptables -F
mahmoud@VM:~$ sudo iptables -I INPUT 1 -p tcp -s 192.168.56.11 --dport 22 -j DROP
mahmoud@VM:~$ sudo iptables -I INPUT 2 -p tcp -s 192.168.56.11 --dport 80 -j DROP
mahmoud@VM:~$ sudo iptables -L INPUT -n --line-numbers -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source               destination
1      0      0 DROP      tcp  --  *      *       192.168.56.11        0.0.0.0/0          tcp dpt:22
2      0      0 DROP      tcp  --  *      *       192.168.56.11        0.0.0.0/0          tcp dpt:80
```

FIGURE 22 – Règles iptables sur A bloquant les connexions entrantes (ports 22 et 80) depuis B.

```
mahmoud@VM:~$ ssh mahmoud@192.168.56.10
^C
mahmoud@VM:~$ curl http://192.168.56.10
^C
```

FIGURE 23 – Échec des accès SSH et HTTP directs depuis B vers A.

```
mahmoud@VM:~$ ssh -R 8080:localhost:80 mahmoud@192.168.56.11
mahmoud@192.168.56.11's password:
Welcome to Ubuntu 16.04.2 LTS (GNU/Linux 4.8.0-36-generic i686)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

1 package can be updated.
0 updates are security updates.

Last login: Mon Apr 13 06:26:11 2026 from 192.168.56.10
mahmoud@VM:~$
```

FIGURE 24 – Établissement du reverse SSH tunnel avec `-R 8080:localhost:80` depuis A.

```
mahmoud@VM:~$ curl http://localhost:8080

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2014-03-19
    See: https://launchpad.net/bugs/1288690
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
  * {
    margin: 0px 0px 0px 0px;
    padding: 0px 0px 0px 0px;
  }

  body, html {
    padding: 3px 3px 3px 3px;
    background-color: #D8DBE2;
```

FIGURE 25 – Accès réussi à la page Apache de A depuis B via `http://localhost:8080`.

6. Conclusion

Ce TP a permis de maîtriser deux outils complémentaires de filtrage sous Linux : `iptables` pour une configuration rapide en espace utilisateur, et Netfilter/LKM pour un filtrage intégré au noyau. Les tâches 3 et 4 ont mis en évidence une limite fondamentale du filtrage statique IP/port : il est contournable dès qu'un canal chiffré (SSH) est autorisé. Une sécurité robuste nécessite une inspection applicative (DPI) et un contrôle strict des connexions chiffrées.